Bike-Rental Forecast Dashboard

Foundations of Data Science - Final Project, Option B (Classical Data Analysis)

Nitzan Wainshtein (209086263), Aviv Oz (207543927)

TODO: submission date

DRAFT. Numbers are taken from the saved results in `outputs/` and `config.json`. The team must review, rewrite in its own words where needed, and fill the TODO items before submission.

1. Application and design

1.1 Task and user

An operations analyst wants to anticipate busy hours and judge which forecasts can be relied on. The dashboard replays historical forecasts: the analyst picks an hour from the final-test period (1 Aug - 31 Dec 2012), obtains the forecast that could have been issued at the start of that hour, and only then reveals the recorded total `cnt`. The target is the system-wide number of rentals during one hour; no station-level allocation is attempted.

Target, availability and excluded fields. One row of `hour.csv` is one hour: the row stamped 08:00 covers 08:00-08:59, and `cnt` (= `casual` + `registered`) is the number of rentals started in it. A forecast is issued at the start of the hour, t. It may use (i) the calendar fields of t (hour, weekday, month, holiday, working day), which are known in advance, and (ii) the count and weather of hours $s < t$ that have already ended. We assume that a completed hour's count and weather are available immediately. Excluded: `casual` and `registered` (their sum is the outcome), `instant` (a row number), and `season`, `yr`, `weathersit` and `atemp`, which are not among the required inputs; the target hour's own `weathersit`, `temp`, `hum` and `windspeed` would in any case only be known after the hour, so only the previous hour's weather is used. `dteday` and `hr` are used only to build the timestamp.

Screen	What the analyst does and sees

1.2 Screens

Screen	What the analyst does and sees
Data overview	Tiles: observed of expected hours, missing hours by period, duplicate and invalid rows, longest gap. *The daily rhythm* by day type. *Timestamp audit*. *Demand shifted upward* (training vs validation bins, TV). *Explore a date range*: selector driving observed / missing / excluded counts, hourly `cnt` with gaps preserved, average by hour and working-day status with group sizes, source rows. Days with missing hours; field definitions; CSV downloads.
Forecast one hour	Control row: test date, observed hour, model, *Forecast*, *Reveal recorded count* (disabled until a forecast exists). Result card: point forecast, model setting, 80% and 95% intervals on a bar (or "Intervals unavailable for this model"); the recorded count stays hidden until *Reveal*, which adds it with the signed and absolute error and whether it falls inside the intervals. Uncertainty card: Bayesian variance decomposition and agreement with the saved prediction. *Inputs used*: calendar tiles and a recent-history table with values, source hours, Observed / Missing status and training-median replacement. Changing the date, hour or model clears the forecast and reveal.
Compare models	Tiles: default model, test MAE and RMSE, change against the seasonal baseline. *Test MAE by model* bars. Full final-test table (setting, validation MAE, test MAE / RMSE, training time, default marked); Poisson probability score. *Forecasts against recorded counts* for a chosen model and test date range; tie rule; validation candidates; CSV downloads.
Inspect model behaviour	Tabs in order: *Interval coverage* (tiles, 80% coverage by hour, tables by hour and working-day status); *Residuals by hour*; *Largest errors* (a selected row opens that hour); *Regularisation* (each Lasso penalty, two plots, coefficients and intercept for a selected penalty, comparison across penalties); *History experiment*.

Figures 1 to 4 show the four screens at desktop width.

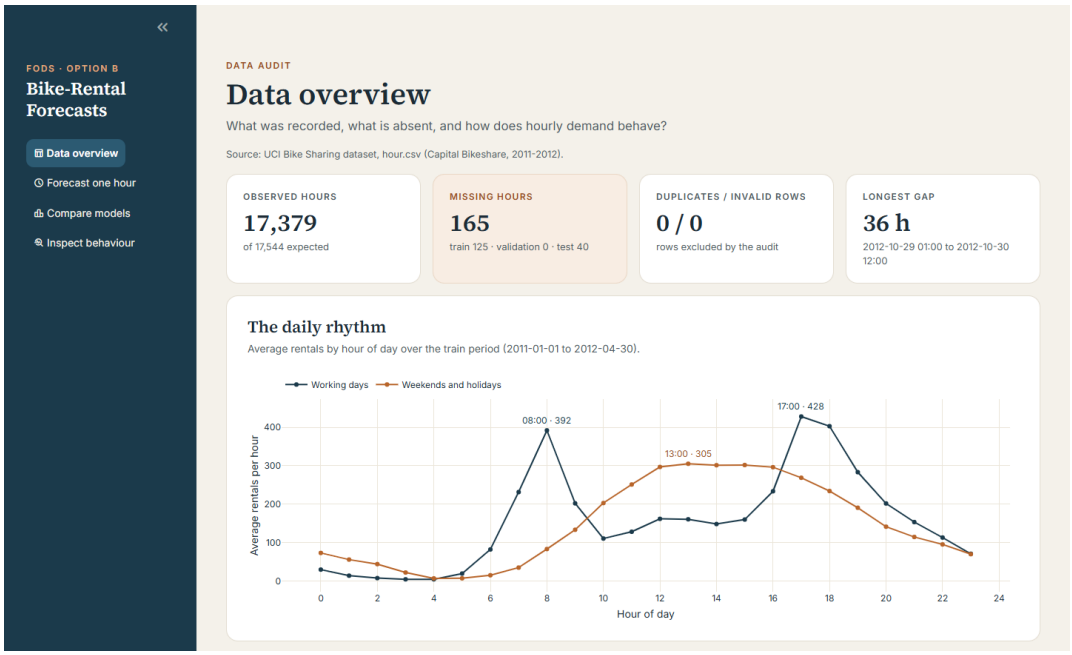


Figure 1. Data overview screen, top part (the full screen is in `docs/screenshots/data_overview.png`). Notice the audit summary (17,379 of 17,544 expected hours observed, 165 missing) and the two working-day peaks in the daily rhythm.

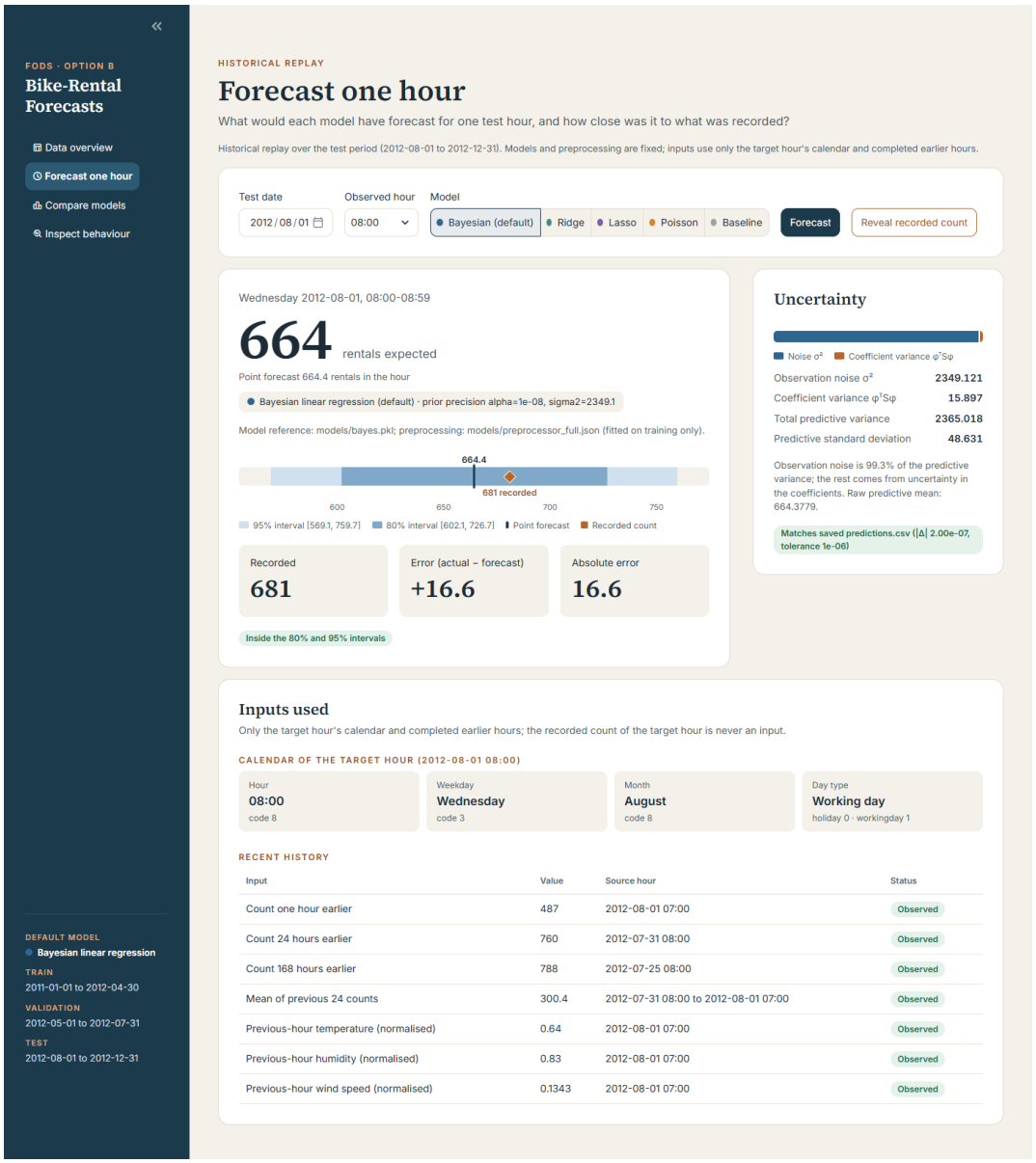


Figure 2. Forecast one hour after *Reveal*, for 2012-08-01 08:00 with the default model. Notice the forecast of 664 rentals, the recorded 681 inside both intervals, and that observation noise dominates the predictive variance.

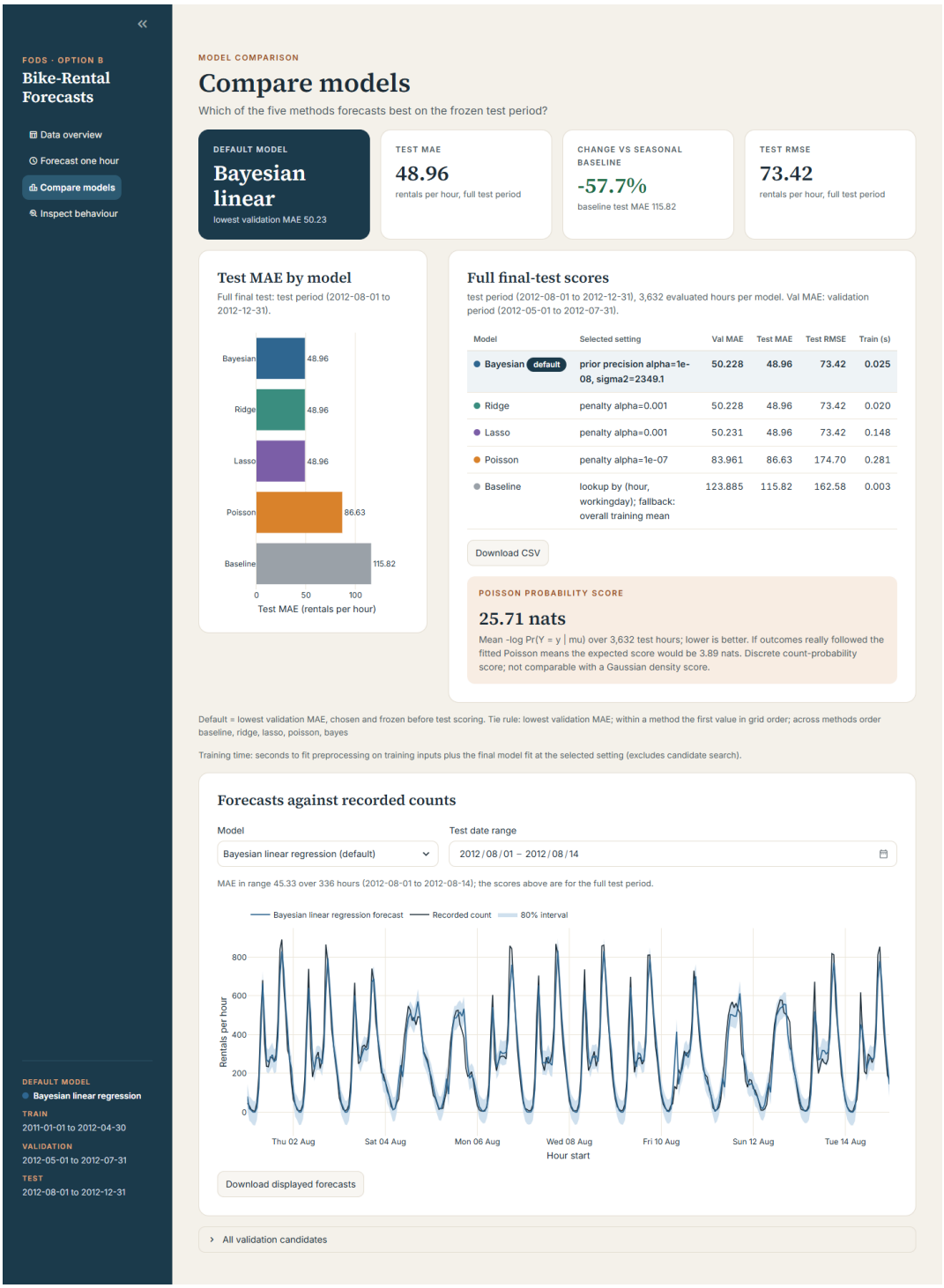


Figure 3. Compare models screen. Notice the three linear-Gaussian models level at a test MAE near 48.96, well below Poisson and the seasonal baseline, and the default model marked in the table.

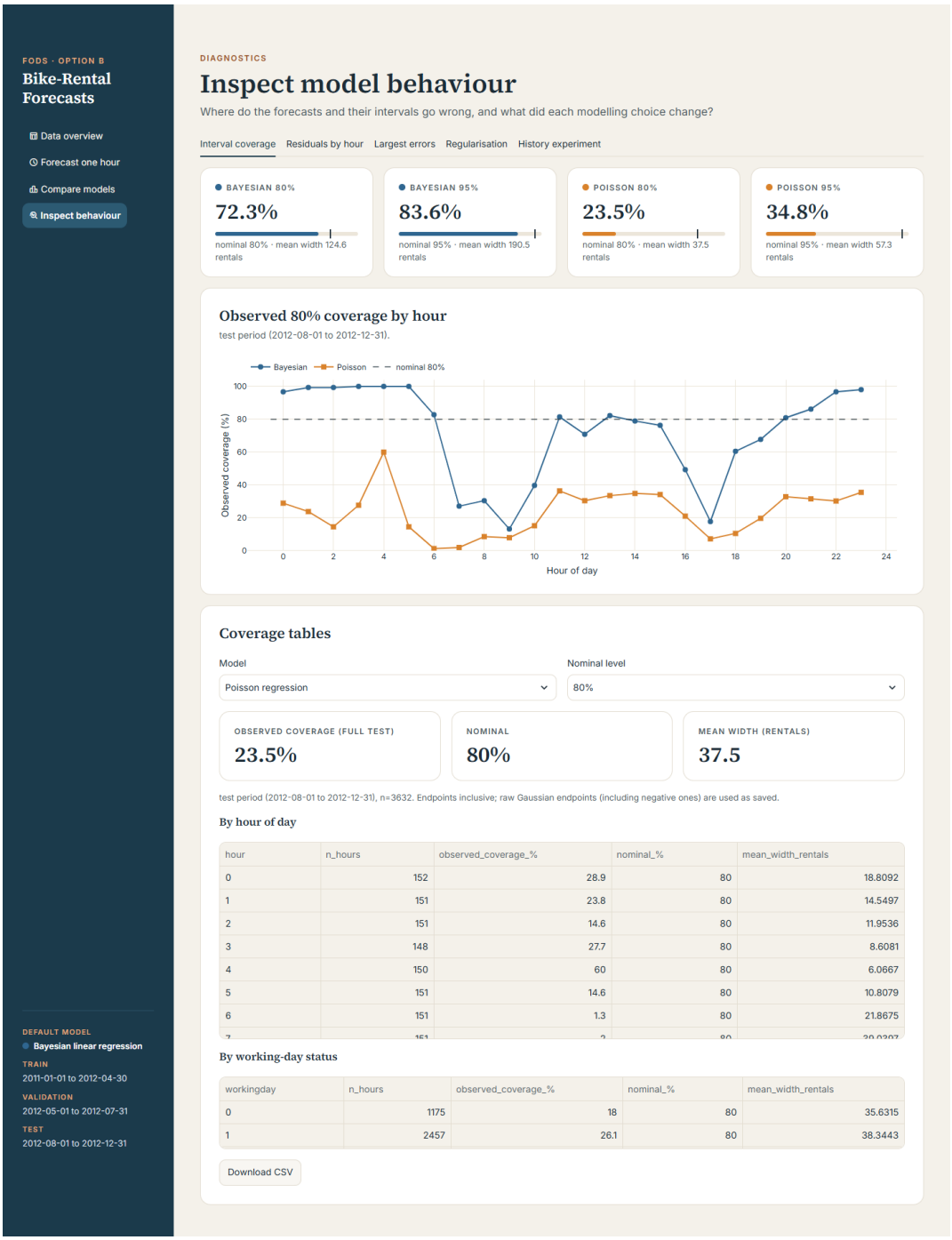


Figure 4. Inspect model behaviour, *Interval coverage* tab. Notice that observed coverage is below nominal for both models at both levels, and that Bayesian 80% coverage falls sharply at commuter hours.

1.3 Code organisation

Module	Responsibility	Key interface
`data.py`	Read `hour.csv`, build timestamps, audit duplicates/invalid rows/missing hours, assign periods	`audit(raw) -> AuditResult(clean, missing_hours, excluded, audit_table)`
`features.py`	Calendar and historical inputs by exact timestamp lookup; `Preprocessor` (training medians, one-hot, scaling)	`build_raw_features(clean, targets) -> DataFrame`; `Preprocessor.fit / transform / save / load`
`models.py`	The five methods; intervals; scores	`predict_dist(raw, X) -> {raw_mean, point, lo80, hi80, lo95, hi95, noise_var, coef_var, total_var}`
`train_evaluate.py`	Fit on training, select on validation, freeze, replay validation/test, write all results	`fit_select(clean)`; `main()`
`app.py`	The four screens; loads saved objects, never refits	-
`ui.py`	Presentation only: design tokens, one colour per model, shared Plotly template, HTML components	`inject_css()`; `chart(fig)`; `interval_bar(model, d, actual)`
`checks/`	32 executable checks (pytest)	-

Input construction never receives the target outcome: `build_raw_features` reads only the target's calendar fields and rows at earlier timestamps; the recorded `cnt` is attached afterwards for scoring (`train_evaluate.py`) or on *Reveal* (`app.py`).

1.4 Two design choices against alternatives

1. **Exact timestamp lookup instead of row shifting.** `df.shift(1)` is simpler but, when 05:00 is absent, gives the 04:00 count as "previous hour" for 06:00. We look up $t - 1\,\text{h}$ in the time index, so an absent hour becomes a flagged missing input. Cost: a slightly slower builder for single hours (the training script uses a vectorised equivalent that a check proves identical).
2. **The app rebuilds inputs and re-applies the saved model, rather than displaying stored predictions.** Reading `predictions.csv` would be simpler, but it would not demonstrate that the deployed path (input builder + saved preprocessing + saved model) works. We rebuild and then compare with the saved value; the Forecast screen shows the difference (about 2e-7 rentals for 2012-08-01 08:00, within the 1e-6 tolerance).

1.5 Worked replay: 2012-08-01 08:00, default model (Bayesian linear regression)

Calendar inputs: hour 8, Wednesday (weekday 3), month 8, holiday 0, workingday 1.

Input	Value	Source
Count one hour earlier	487	2012-08-01 07:00
Count 24 hours earlier	760	2012-07-31 08:00
Count 168 hours earlier	788	2012-07-25 08:00
Mean of previous 24 counts	300.4167	2012-07-31 08:00 to 2012-08-01 07:00
Previous-hour temp / hum / windspeed	0.64 / 0.83 / 0.1343	2012-08-01 07:00

No input was missing. Forecast: **664.4** rentals; 80% interval [602.1, 726.7]; 95% interval [569.1, 759.7]. After *Reveal*: recorded **681**, signed error +16.6, absolute error 16.6; both intervals cover the outcome.

The 681 is not used for this forecast; it becomes the previous-hour input for 09:00. For comparison the seasonal baseline gives 392.0 (working-day 08:00 training mean) and Poisson 1018.4.

2. Data and modelling

2.1 Audit findings

`hour.csv` has 17,379 rows, no duplicate timestamps and no invalid counts. Compared with a complete hourly calendar (17,544 hours), **165 hours are absent**: 125 in training, 0 in validation, 40 in test. The test gap includes 36 consecutive hours from 2012-10-29 01:00 to 2012-10-30 12:00; the others are isolated early-morning hours. No observed hour has `cnt = 0`. Absent hours are not scored and are never converted to zero; when an absent hour is a source for a lag, that input is missing.

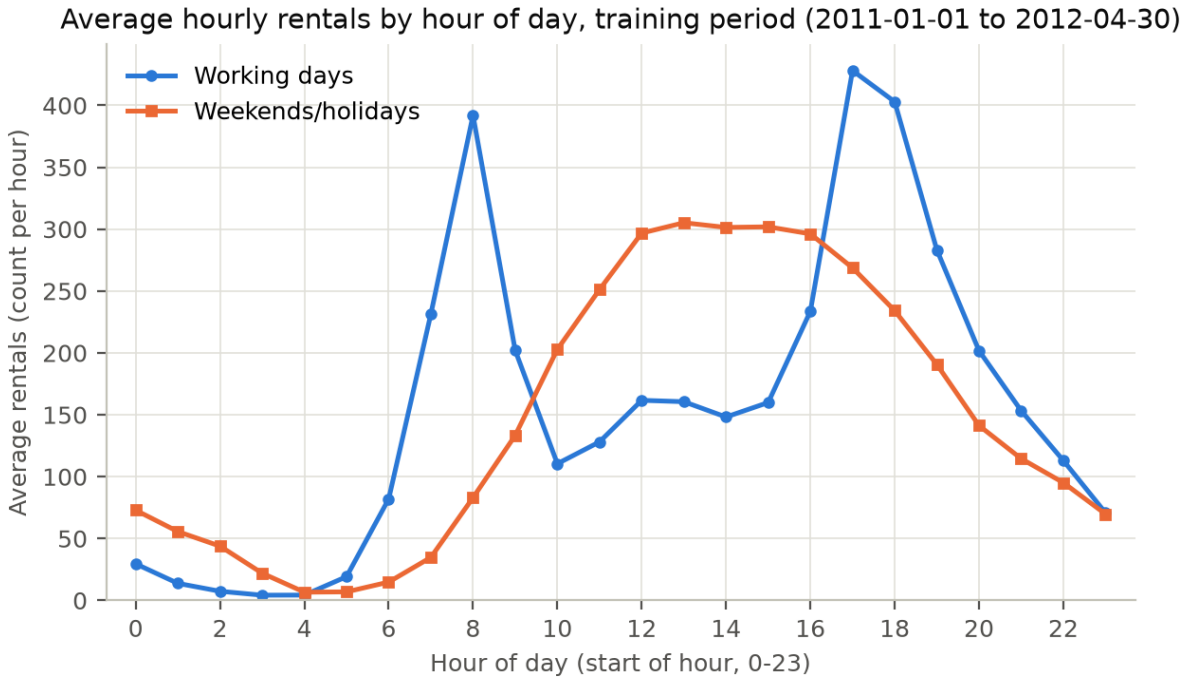


Figure 5. Average training-period rentals by hour of day. Notice the two commuter peaks on working days (08:00 and 17:00-18:00) against the single broad midday peak on weekends and holidays.

2.2 Periods

Period	Dates (inclusive)	Observed target hours
Training	2011-01-01 - 2012-04-30	11,539
Validation	2012-05-01 - 2012-07-31	2,208
Final test	2012-08-01 - 2012-12-31	3,632

Learned parameters (medians, encodings, scaling, coefficients, posterior, selected settings, default) are fixed after training/validation. History, however, advances during replay: forecasting 09:00 uses the recorded 08:00 count, without changing any coefficient.

2.3 Inputs and preprocessing

Calendar: hour (24), weekday (7, dataset convention Sunday = 0), month (12) one-hot; holiday and workingday binary. History: counts at $t-1$, $t-24$, $t-168$ h; mean of the 24 counts $t-24, \dots, t-1$ (missing unless all 24 exist); temp, hum, windspeed at $t-1$ (normalised dataset units). Each of these seven has a missing flag.

All fitted on training only: medians (previous-hour count 117, previous-day 117, previous-week 116, 24-hour mean 171.04, temp 0.44, hum 0.62, windspeed 0.194); standardisation (mean/sd after imputation); category lists. Unseen categories map to an all-zero block; a constant column would be centred only. Result: 59 columns. `checks/test_frozen.py` reruns fitting with random test-period values and confirms identical preprocessing.

Missing-input rule. A historical input is missing when its source hour is absent from the data (or, for the 24-hour mean, when any of the 24 hours is absent). It is then replaced by the training median and its flag is set to 1; the flag records missingness before replacement, so the model can treat a replaced value differently from an observed one. In the test period this affects the hours after the 36-hour gap of 29-30 October 2012 (for example 2012-10-30 13:00, whose previous-hour count is missing).

2.4 Model definitions and selected settings

Method	Definition	Candidates	Selected	Validation MAE
Seasonal baseline	training mean of `cnt` per (hour, workingday); fallback overall mean	-	lookup	123.88
Ridge	$b + x^\top w$, L2 penalty, clipped at 0	$\alpha \in \{0.001, 0.01, \dots, 1000\}$	0.001	50.228
Lasso	$b + x^\top w$, L1 penalty, clipped at 0	10 values 0.001 - 30	0.001	50.231
Poisson	$\mu = \exp(b + x^\top w)$, L2 penalty	1e-7 - 0.1	1e-7	83.96
Bayesian linear	$\beta \sim \mathcal{N}(0, \alpha^{-1} I)$, $y \sim \mathcal{N}(\Phi\beta, \sigma^2 I)$	$\alpha \in \{1\text{e-}8, \dots, 1\}$; σ^2 by training fixed point	$\alpha =$ 1e-8, $\sigma^2 = 2349.1$	50.228

For the Bayesian model, σ^2 is estimated on training data by alternating between the posterior mean and the mean squared training residual until convergence. Posterior: $A = \alpha I + \Phi^\top \Phi/\sigma^2$, m solves $Am = \Phi^\top y/\sigma^2$, S solves $AS = I$.

This σ^2 is a **plug-in estimate**, not a fully Bayesian treatment of the noise: everything after it (posterior, predictive variance, intervals) is conditional on $\sigma^2 = 2349.1$ as if it were known. It omits uncertainty in σ^2 itself and makes no degrees-of-freedom correction. We chose it because it is simple, uses training data only, and is easy to verify. The omission is small here: with $n = 11{,}539$ training hours and $p = 60$ parameters, the corrected value $\mathrm{RSS}/(n-p)$ is 2361.4, which widens every interval by only 0.26% (validation 80% coverage would move from 71.2% to 71.24%). A fuller treatment, a prior on σ^2 giving Student-t predictive intervals, would barely change the width at this sample size. So the degrees-of-freedom correction is far too small to explain the undercoverage in section 3.3. The strong hour-dependent coverage pattern suggests that the constant-variance assumption is an important contributor, although other misspecification (errors in the mean around the peaks, dependence between hours) may also contribute.

Ridge, Lasso and Bayesian validation MAE decrease monotonically as the penalty/prior precision decreases and flatten at the bottom of the grid (Ridge 50.2283 at 0.001 and 50.2284 at 0.01). With 11,539 training hours and 59 columns, little regularisation is needed for prediction.

Default. Lowest validation MAE: Bayesian 50.22829, Ridge 50.22830 (practically identical; with a near-flat prior the posterior mean approximates the least-squares fit). The tie rule was not needed. The default was frozen in `config.json` before test scoring. The difference (0.00001 rentals per hour) has no practical meaning: Bayesian regression is the default because the stated rule picks it, not because it forecasts better than Ridge, and the dashboard says so next to the comparison table.

2.5 One Bayesian predictive calculation (actual input)

For 2012-08-01 08:00 (section 1.5): $\mu_* = \phi^\top m = 664.378$; observation-noise variance $\sigma^2 = 2349.121$; coefficient-uncertainty variance $\phi^\top S \phi = 15.897$; total $v_* = 2365.018$ (sd 48.63). 80% interval $664.378 \pm 1.28155 \times 48.63 = [602.05, 726.70]$. Almost all predictive uncertainty is observation noise: with 11,539 training hours the coefficients are well determined.

3. Results

3.1 Five-model point errors (final test, 3,632 hours)

App: Compare models (tiles, "Test MAE by model", "Full final-test scores", "Forecasts against recorded counts"). Saved: `outputs/metrics.csv` *(*`table = point_scores`*),* `outputs/predictions.csv`*.*

Model	MAE	RMSE	Validation MAE	Training time (s)
Seasonal baseline	115.82	162.58	123.88	0.003
Ridge	48.955	73.415	50.228	0.020
Lasso	48.961	73.419	50.231	0.148
Poisson	86.63	174.70	83.96	0.281
Bayesian (default)	**48.955**	**73.415**	50.228	0.025

The three linear-Gaussian models reduce MAE by about 58% relative to the baseline. Their test ranking matches validation, but the differences among them (< 0.01 rentals) are negligible; validation selection should not be read as evidence that Bayesian regression forecasts better than Ridge. RMSE/MAE is about 1.5 for these models, indicating occasional large errors (section 3.7). Poisson has the worst RMSE (174.7), worse even than the baseline, because of very large overpredictions at evening peaks.

Test against validation. The default model's test MAE (48.96) is slightly lower than its validation MAE (50.23), so the validation estimate was not optimistic. A likely contributor is the level of demand: validation (May-July 2012) has a mean of 272.8 rentals per hour and 29.5% of hours at 369 or more, while the test period has 250.1 and 25.2%, with November and December averaging 213 and 167. This is a description of the two periods, not a demonstrated cause. Poisson is the only model whose error grew from validation to test (83.96 to 86.63).

Training time is measured identically for every model: seconds to fit the preprocessing on the training inputs plus the final fit at the selected setting; the search over candidate settings is excluded. All five fit in under 0.3 s, so training cost does not separate the models.

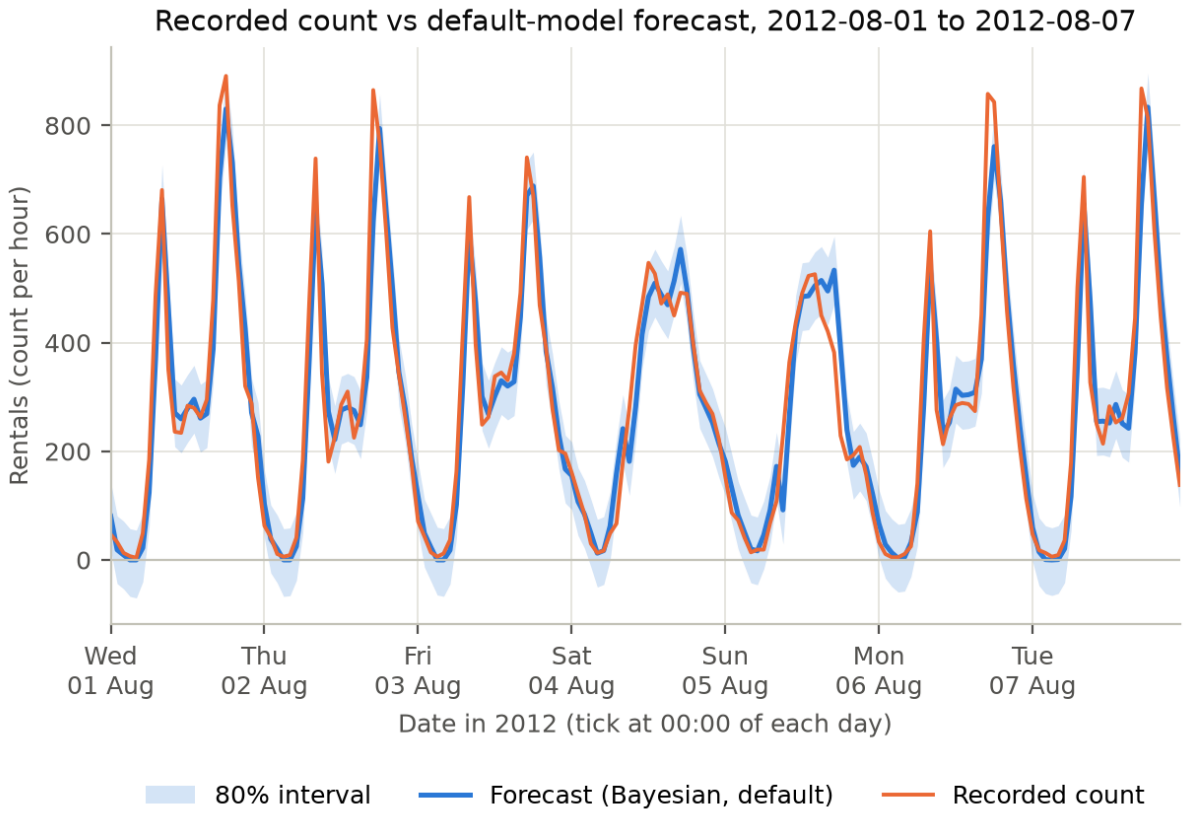


Figure 6. Recorded count and default-model forecast for the first test week, with the 80% interval. Notice that the forecast follows the daily cycle, but at the working-day 08:00 and 17:00 peaks the recorded count is usually above the band, and that the band's lower edge falls below zero overnight.

3.2 Poisson probability score

App: Compare models ("Poisson probability score"). Saved: `outputs/metrics.csv` *(*`table = poisson_log_score`*).*

The Poisson model assumes that, given the inputs, the hour's count follows a Poisson law with mean μ, so its variance equals its mean. (Independence between hours matters for reading the summed log score as a joint likelihood; the per-hour average reported here does not need it.) The selected penalty is the smallest tested, 10^{-7}.

Mean negative log probability of the recorded count under Poisson(μ): **25.71 nats** over 3,632 test hours. If the outcomes really followed Poisson(μ) with the fitted means, the expected score would be the average Poisson entropy, 3.89 nats. The large value reflects both mean errors and a variance ($= \mu$) far too small for the observed variation.

3.3 Interval coverage and width (test)

App: Inspect model behaviour → Interval coverage (tiles, coverage-by-hour chart, coverage tables). Saved: `outputs/metrics.csv` *(*`table = interval_coverage`*), interval endpoints in* `outputs/predictions.csv`*.*

The two constructions differ. The Poisson interval contains the central 80% or 95% of the counts of a Poisson law whose mean is fixed at the fitted μ: it describes only count-to-count variation and ignores uncertainty in μ. The Bayesian interval is $\mu_* \pm z\sqrt{\sigma^2 + \phi^\top S \phi}$: Gaussian noise around the line plus uncertainty in the coefficients.

Model	Nominal	Observed coverage	Mean width
Bayesian	80%	72.3%	124.6
Bayesian	95%	83.6%	190.5
Poisson	80%	23.5%	37.5
Poisson	95%	34.8%	57.3

Both models undercover, and the shortfall is larger than sampling noise. Because errors on the same day are correlated, we resampled whole test days (153 days, 5,000 day-block bootstrap draws, seed 0; script `coverage_bootstrap.py`, output `outputs/coverage_bootstrap.csv`): 95% intervals for the observed coverage are Bayesian 80%: 71.0-73.6%, Bayesian 95%: 82.5-84.8%, Poisson 80%: 21.7-25.2%, Poisson 95%: 32.6-37.0%. None contains its nominal level. Bayesian intervals have nearly constant width (noise variance dominates), while the errors are strongly heteroscedastic: by hour, 80% coverage is 100% at 03:00 (counts near zero, interval far too wide) and only 13.2% at 09:00. Coverage is similar on working days (72.9%) and other days (71.0%). Because the Gaussian approximation ignores that counts are non-negative, about 30% of test hours have a negative lower 95% endpoint (kept as saved). The Gaussian assumptions therefore do not match the test results: the noise is not constant across hours, counts cannot be negative, and the coefficient term is too small (about 0.7% of the variance for 2012-08-01 08:00) to compensate.

Poisson intervals are much narrower (variance $= \mu$) and miss most outcomes: hourly counts are strongly overdispersed relative to Poisson, and the mean itself is biased at peaks. Coverage is worst at commuting hours (80%: 1.3% at 06:00).

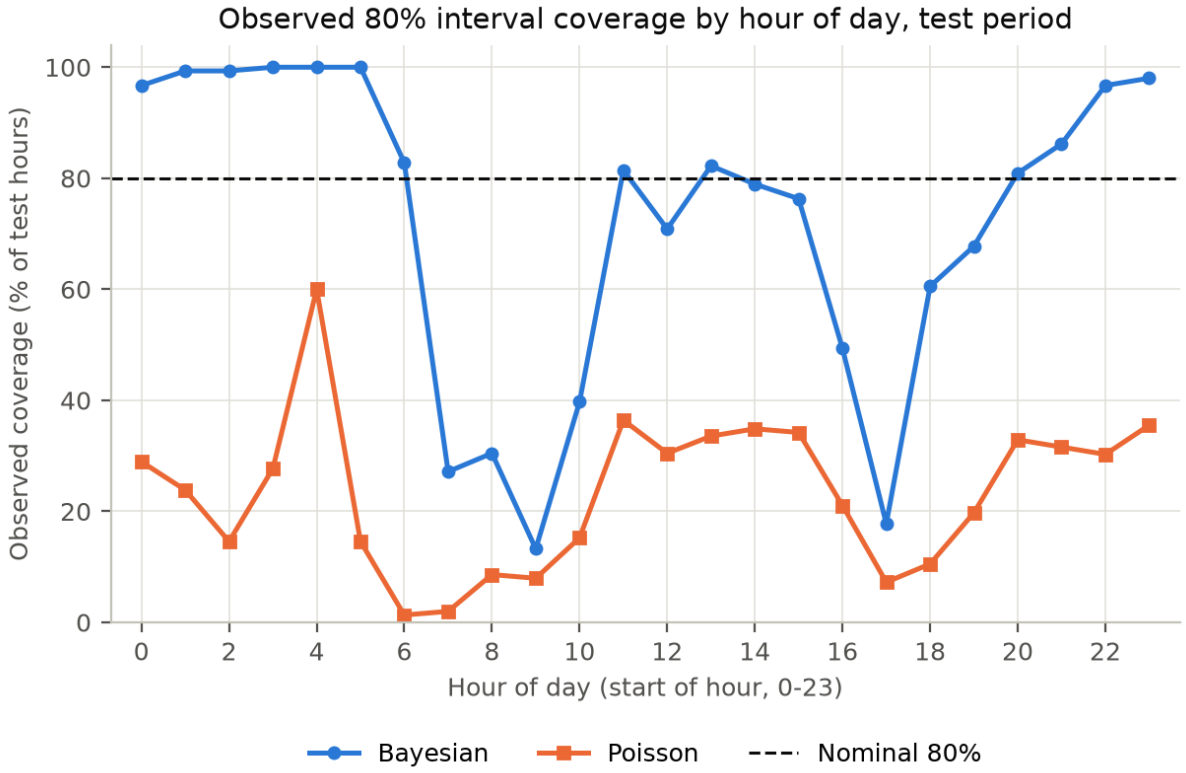


Figure 7. Observed 80% interval coverage by hour of day on the test period. Notice that Bayesian coverage is above the nominal 80% overnight and collapses at commuter hours (13.2% at 09:00), while Poisson stays below 80% in every hour.

3.4 Lasso regularisation

App: Inspect model behaviour → Regularisation (penalty table, two plots, coefficient table for a selected penalty, comparison across penalties). Saved: `outputs/validation_results.csv`, `outputs/lasso_coefficients.csv`.

α	0.001	0.01	0.1	0.3	1	3	10	30
Validation MAE	50.23	50.26	50.73	52.03	54.62	55.75	57.34	69.33
Non-zero coefficients (of 59)	54	52	42	32	14	6	3	3

At the selected 0.001, five coefficients are zero (`hour=03`, `weekday=04`, `month=10` and two weather missing flags that duplicate the previous-hour count flag). At $\alpha = 1$, only peak-hour dummies (06-09, 16, 17, 19, 20), the three count lags and the three weather inputs remain, and MAE rises by 4.4. At $\alpha = 10$ only the three count lags remain. Weekday dummies are removed early, since working-day status and recent counts already carry most of that information. Zero coefficients are properties of this penalised, correlated fit and do not show that a variable is unrelated to rentals. The number of non-zero coefficients is not monotone either: 52 at 0.01 but 53 at 0.03.

Reading the signs. Numeric inputs were standardised on training data, so a coefficient is the change in forecast per one training standard deviation of that input (for the previous-hour count, 146 rentals), holding the other inputs fixed. Hour, weekday and month are fully one-hot encoded next to an intercept, so their coefficients are only meaningful relative to each other. At the selected 0.001: previous-hour count +87.0, week-earlier count +38.9, day-earlier count +17.4, previous-hour temperature +11.6, humidity −7.5, wind speed −3.9, and the 24-hour mean −2.3. The negative sign of the 24-hour mean does not mean that busier recent days reduce demand: the four count inputs are strongly correlated, so the fit shares their effect and one of them can take a small opposite sign. As the penalty grows, the shared effect moves to the inputs that remain: the 24-hour mean is removed by $\alpha = 1$, and the week-earlier coefficient rises from 38.9 to 51.7 at $\alpha = 10$ while the hour and weather terms go to zero. None of these signs is a causal effect.

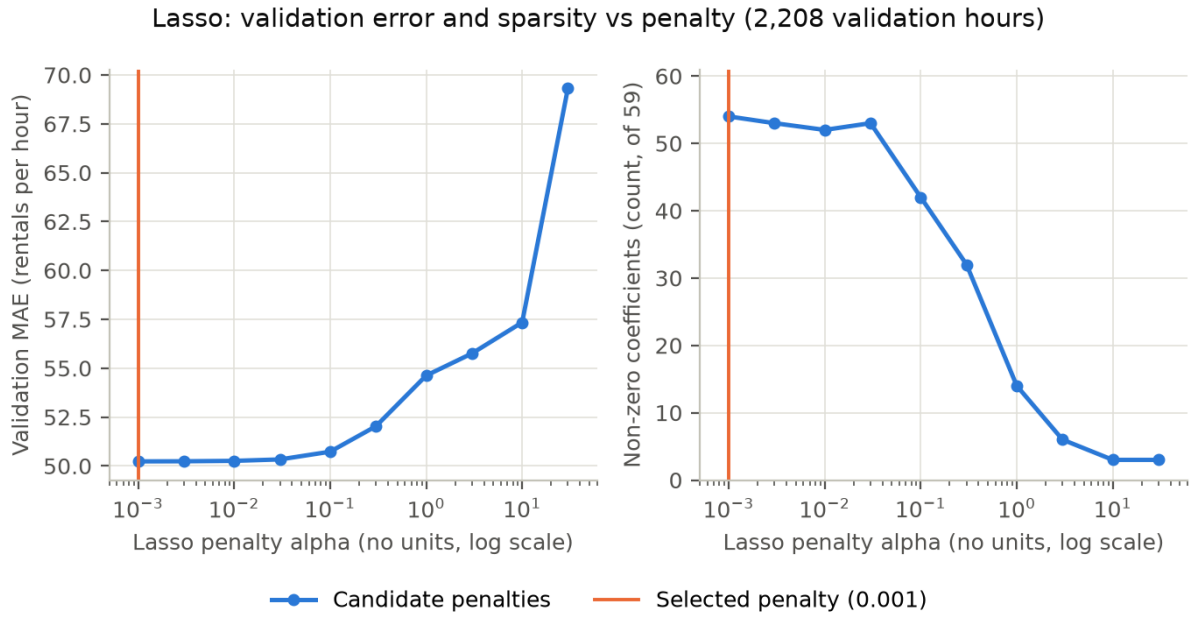


Figure 8. Lasso validation MAE and number of non-zero coefficients against the penalty. Notice that the MAE is almost flat up to a penalty of 0.1 and rises after that, while the model shrinks from 54 non-zero coefficients at the selected 0.001 to 3 at the largest penalties.

3.5 Does historical information help? (validation)

App: Inspect model behaviour → History experiment. Saved: `outputs/validation_results.csv` *(*`experiment = history_*`*).*

Input set	Selected penalty	Hours	Validation MAE
Calendar only	0.001	2,208	115.17
Full required inputs	0.001	2,208	50.23

Calendar-only minus full: **+64.94** rentals per hour. The calendar-only Ridge is close to the seasonal baseline (123.9). The gain comes from recent counts and previous-hour weather together; this experiment does not separate them and does not establish a causal effect.

3.6 Training vs validation count distributions

App: Data overview → "Demand shifted upward". Saved: `outputs/count_distribution.csv`*,* `outputs/count_distribution_tv.json`*.*

Bins from training deciles (edges 0, 7, 23, 49, 81, 116, 154, 200, 263, 369 and an overflow bin). **TV = 0.279.** The largest change is the top bin, 369 and above: 10.0% of training hours vs 29.5% of validation hours. Low-count bins shrink (0-6: 9.2% to 4.0%; 81-115: 10.1% to 3.8%). Validation (May-July 2012) has more high-demand hours than the 16-month training period, which mixes winter months with 2011 levels. This helps explain why the seasonal baseline, which predicts training averages, has validation MAE 123.9, and why the Poisson log link extrapolates poorly. It does not by itself prove a cause.

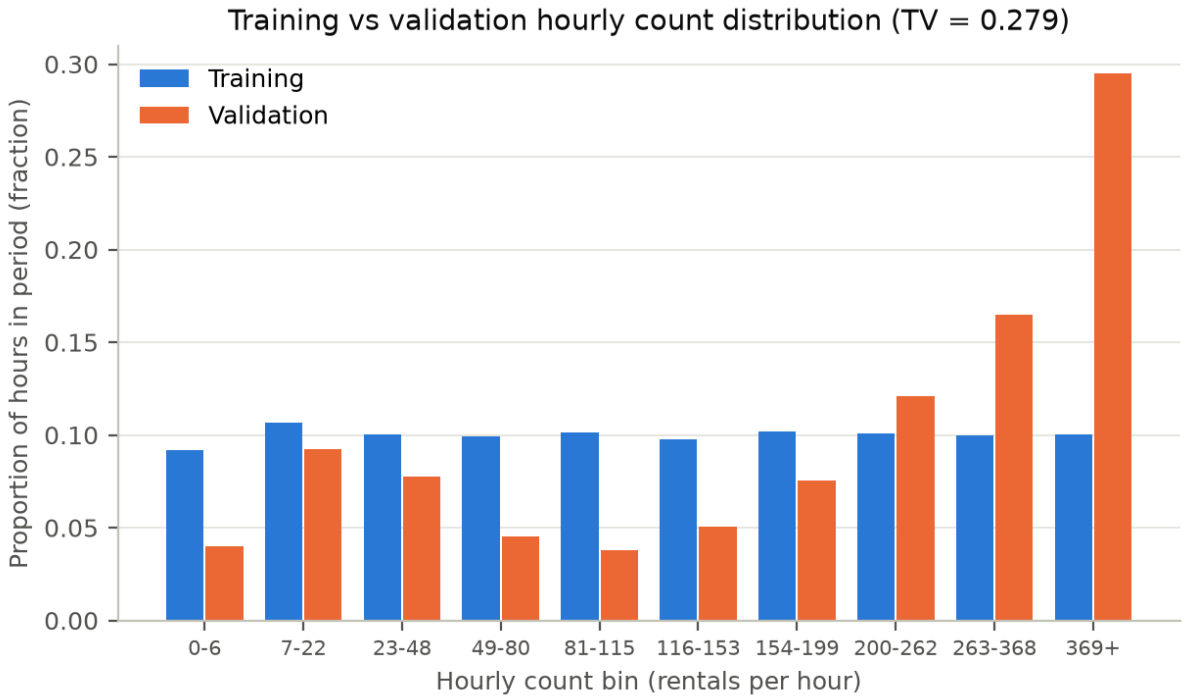


Figure 9. Training and validation proportions of hours in each count bin (TV = 0.279). Notice the much larger share of validation hours in the top bin (29.5% against 10.0%) and the smaller share in every bin below 200 rentals.

3.7 Largest errors and residuals by hour (default model)

App: Inspect model behaviour → Largest errors and Residuals by hour. Saved: `outputs/largest_errors.csv`, `outputs/metrics.csv` *(*`table = residual_by_hour`*).*

The ten largest absolute errors range from 304.6 to 413.8 rentals; 9 of 10 occur at 08:00-09:00 or 16:00-17:00. Mean residual (actual - forecast) by hour:

Hour	06	07	08	09	16	17	18	19
Mean residual	+20.8	+50.1	+45.9	-45.4	+22.2	+36.3	-19.2	-34.3

The pattern is systematic: underprediction as peaks build (07-08, 16-17) and overprediction just after (09, 18-19).

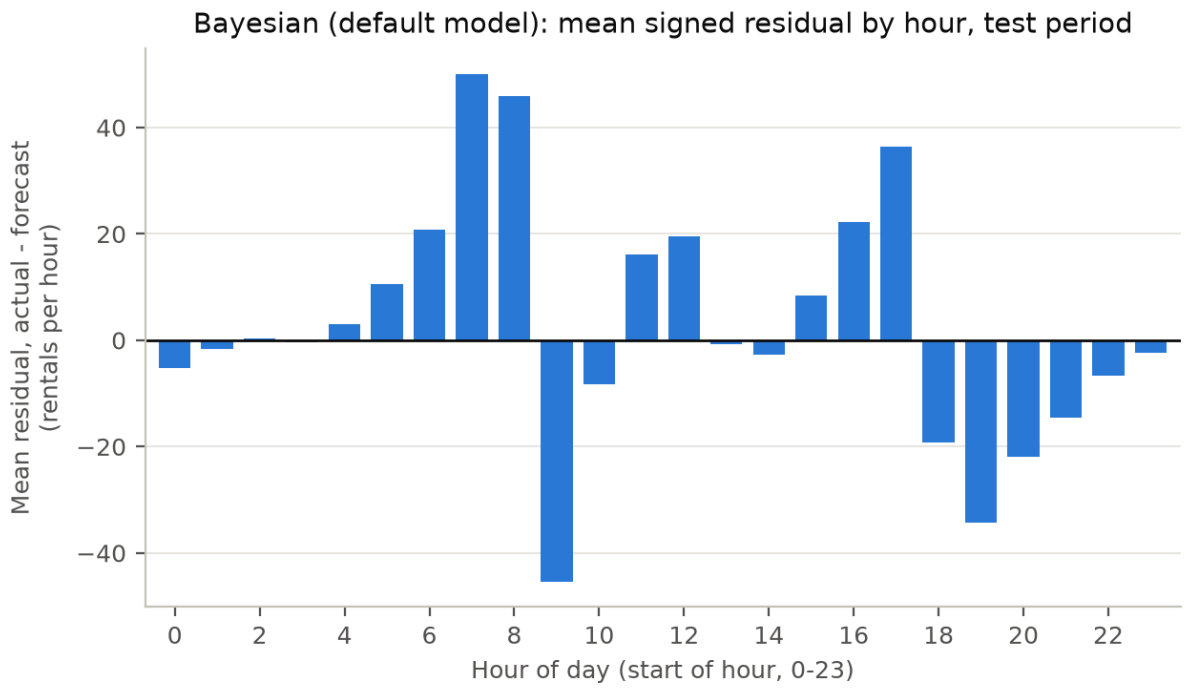


Figure 10. Default-model mean signed residual (actual minus forecast) by hour on the test period. Notice the positive bars as the commuter peaks build (07-08 and 16-17) and the negative bars just after them (09 and 18-19): the errors around the peaks are systematic.

4. Limitations and improvements

4.1 Three large-error cases

1. **2012-10-02 08:00** - actual 134, forecast 547.8, error -413.8. Inputs: previous hour 260, 24 h earlier 784, week earlier 805, previous-hour humidity 0.83. The recorded `weathersit` for 06:00-09:00 is 3 (light rain/snow category) and counts fell further to 86 at 09:00. *Observed:* the model was anchored on the strong same-hour counts of the previous day and week. *Hypothesis:* the weather recorded in that hour suppressed demand, which the previous-hour weather inputs (temp, hum, wind; `weathersit` is not an input) do not capture.
2. **2012-09-06 09:00** - actual 123, forecast 473.8, error -350.8. The previous hour was 646 (a normal peak); `weathersit` changed from 2 to 3 at 08:00 and humidity rose to 0.94 at 09:00. *Observed:* a collapse within one hour that no completed-hour input could reveal. *Hypothesis:* onset of rain.
3. **2012-10-09 08:00** (a working Tuesday) - actual 713, forecast 365.8, error +347.2. Previous hour 362; 24 hours earlier 397, but that hour was a public holiday (2012-10-08, `holiday = 1`); 168 hours earlier only 134, which is exactly case 1, the rain-affected 2012-10-02 08:00. *Observed:* two of the three count lags came from unusual hours (a holiday and a weather-suppressed morning), and both pulled the forecast down. *Hypothesis:* the lags carry an unusual hour forward as if it were typical, so one disrupted hour can cause a second large error a day or a week later. This case also fits the systematic underprediction at 08:00 (mean residual +45.9).

In all three cases no historical input was missing, so the errors do not come from the missing-input rule. The weather explanations rely on the dataset's own `weathersit` and humidity columns; no outside event is assumed.

4.2 Failure modes and proposed changes

- **Peak-shape bias.** Hour and working-day effects are additive, so the 08:00 / 17:00 commuter peaks (working days only) cannot be represented separately from weekend midday peaks. *Change:* add hour × workingday interaction columns in `features.py` (`Preprocessor`), refit Ridge/Lasso/Bayesian, and compare validation MAE and residuals by hour with the current fit, before looking at test.
- **Constant-variance and Poisson intervals.** Bayesian intervals have the same width at 03:00 and at 08:00; Poisson variance is far too small. *Change:* model variance by hour group (or fit on $\log(1 + \text{cnt})$ and back-transform), and replace Poisson by a negative binomial with an estimated dispersion; evaluate with validation coverage/width by hour (`metrics.csv` interval tables).
- **Unusual hours carried forward by the lags** (cases 1 and 3). *Change:* in `features.py`, add flags for whether the 24-hour and 168-hour source hours were holidays or had `weathersit` ≥ 3 (both known once those hours have ended), so the model can discount them; compare validation MAE on the hours whose lag sources are flagged, against the current fit.
- **Poisson log-link extrapolation.** Standardised lag counts enter $\log \mu$ linearly, so higher counts multiply the forecast. *Change:* use $\log(1 + x)$ of lag counts x as Poisson inputs; test by validation MAE and probability score.

4.3 Assumptions and correctness checks executed

`python -m pytest checks` - 32 passed (record in `CHECKS.md`). They establish:

- time availability: changing the target hour's count/weather or any later hour leaves inputs unchanged; changing an eligible earlier count changes its input; a missing $t - 1$ is flagged rather than shifted; recorded zero kept; 24-hour mean 12.5 for counts $1, \ldots, 24$ and missing when one hour is removed; the brief's 08:00 example inputs reproduced;
- frozen learning: random test values leave preprocessing, coefficients, validation table and default unchanged; reloaded models reproduce `predictions.csv`;
- numerics: MAE 2 / RMSE 2.160 example; coverage 2/3 with an endpoint outcome; Poisson quantiles against the CDF; Poisson log score against the formula; Bayesian example ($m = (1.6, 1.2)$, mean 4, coefficient variance 2, total 3, interval [1.78, 6.22]) and agreement with an explicit inverse;
- the app: all screens render; Forecast hides the outcome until Reveal; the forecast agrees with the saved prediction; switching model clears the result; baseline intervals show as unavailable.

Assumptions: completed-hour counts and weather become available immediately; observations are treated as exchangeable within the Gaussian/Poisson likelihoods, although hourly counts are autocorrelated, so interval statements are model-based; the test period is a single autumn, so its scores need not represent other seasons.

References

Fanaee-T, H. and Gama, J. (2013). Event labeling combining ensemble detectors and background knowledge. *Progress in Artificial Intelligence* 2, 113-127. UCI Bike Sharing Dataset.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*, Secs. 3.1-3.3. Springer.

Hastie, T., Tibshirani, R. and Wainwright, M. (2015). *Statistical Learning with Sparsity*, Ch. 2. CRC Press.

scikit-learn documentation: Ridge, Lasso, PoissonRegressor. SciPy documentation: `scipy.stats.poisson`. Streamlit documentation.